TRƯỜNG ĐẠI HỌC VĂN HIẾN

Thành Nhân trước thành Danh

MÔN HỆ ĐIỀU HÀNH

Giảng viên: ThS. Ngô Quốc Huy

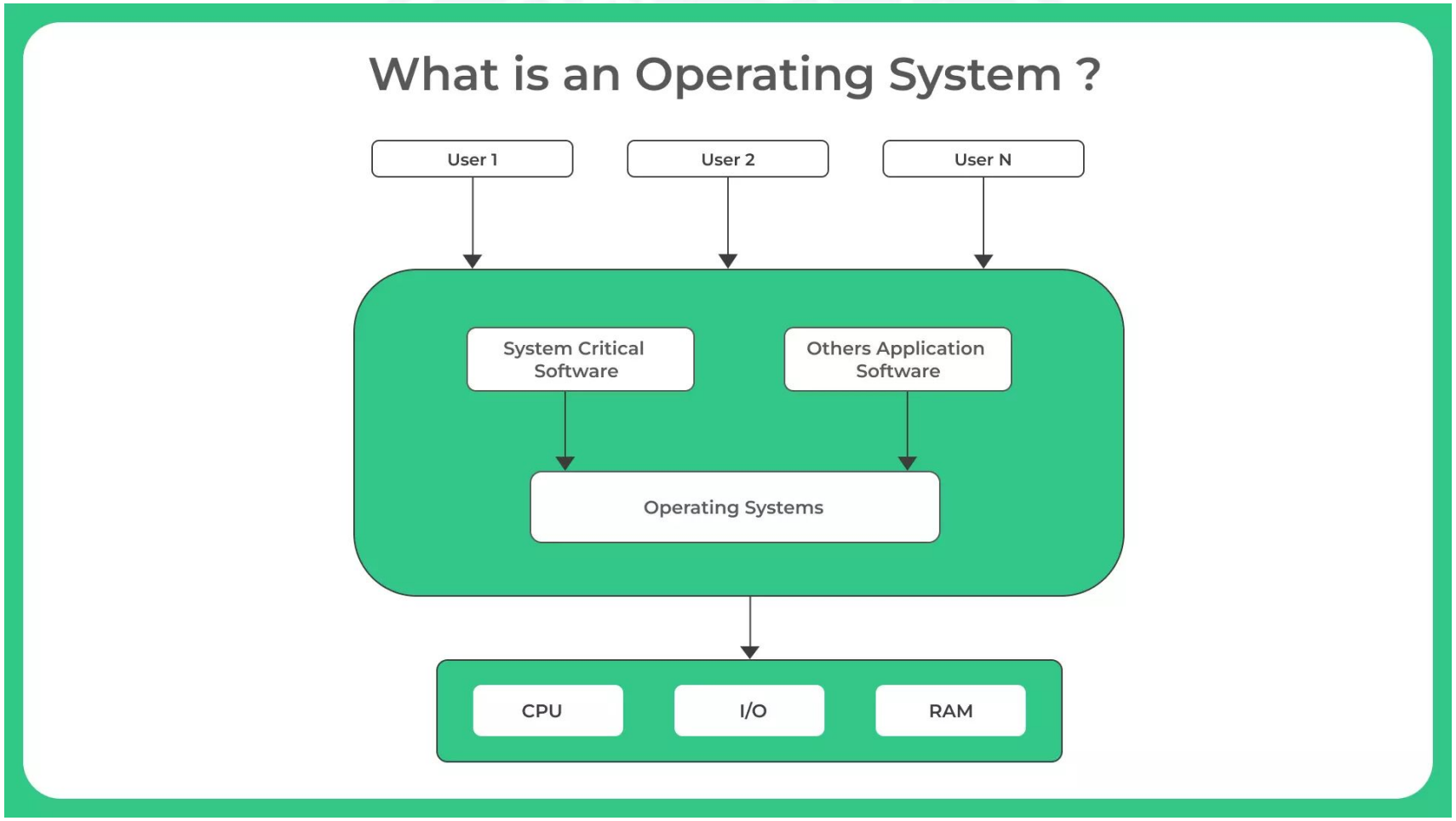
Thời lượng: 30 tiết

CHƯƠNG 1: TỔNG QUAN VỀ HỆ ĐIỀU HÀNH

NỘI DUNG

- Hệ điều hành là gì
- Lịch sử phát triển
- System calls
- Cấu trúc hệ điều hành
- Các chương trình hệ thống
- Boot hệ thống

Hệ điều hành là gì?



Hệ điều hành là gì?

- Nhiệm vụ chung do hệ điều hành thực hiện:
  - Xử lý giao diện người dùng
  - Xử lý đầu vào/đầu ra
  - Quản lý tiến trình
  - Quản lý tập tin
  - Quản lý bộ nhớ
  - Bảo mật
  - Các thiết bị khác điều khiển như máy in, thẻ SD

Hệ điều hành là gì?

- Hệ điều hành phổ biến
  - Windows
  - MacOS
  - Linux
  - Android
  - iOS

Lịch sử phát triển

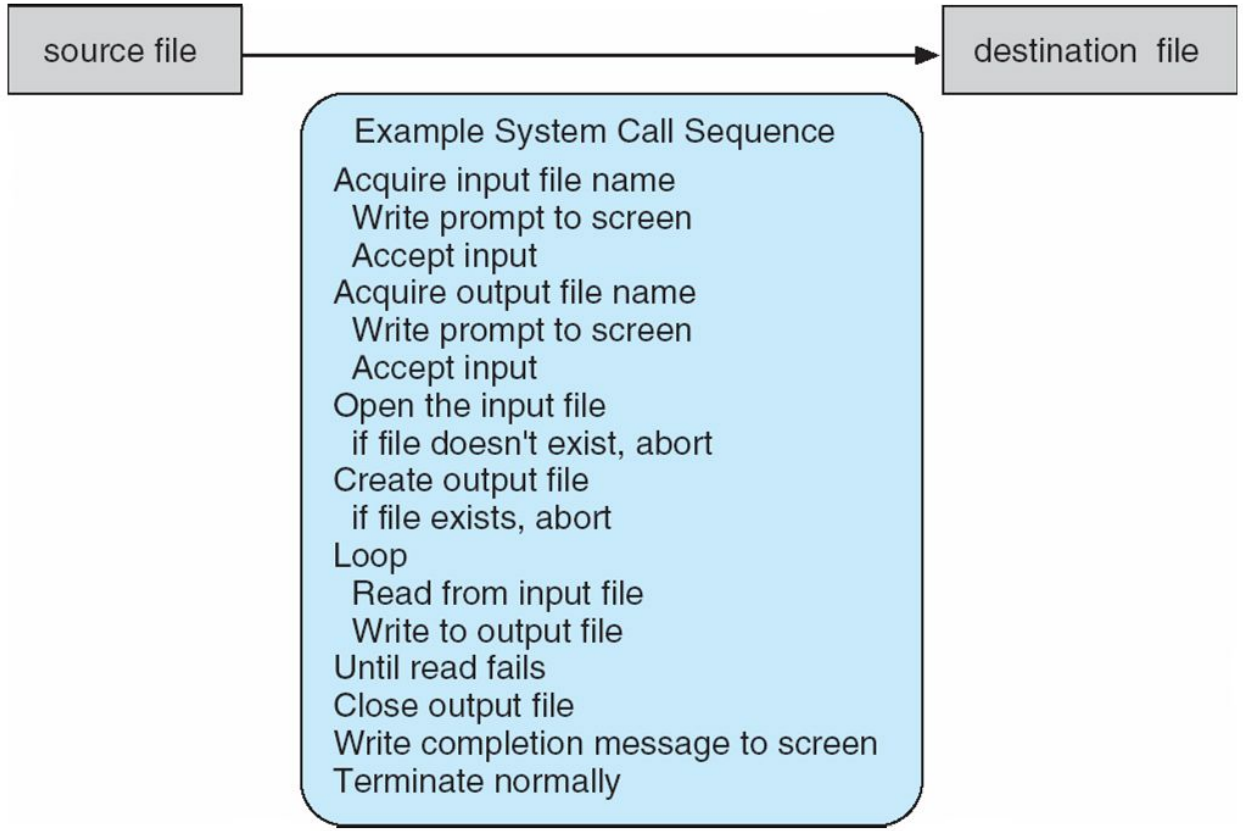
Thế hệ	Năm	Thiết bị điện tử được sử dụng	Các loại thiết bị hệ điều hành
Thứ nhất	1945-55	Ống chân không	Bảng cắm
Thứ hai	1955-65	Bán dẫn	Batch Systems
Thứ ba	1965-80	Integrated Circuits(IC)	Đa chương trình
Thứ tư	Từ 1980	Large Scale Integration	PC

System calls

- Giao diện lập trình với các dịch vụ được cung cấp bởi hệ điều hành
- Thường được viết bằng một ngôn ngữ cấp cao (C hoặc C ++)
- Được truy cập bởi các chương trình thông qua một giao diện lập trình ứng dụng cấp cao (Application Programming Interface - API)
- Ba API phổ biến nhất là:
  - Win32 API cho Windows
  - POSIX API cho các hệ thống POSIX (bao gồm hầu như tất cả các phiên bản của UNIX, Linux, và Mac OS X)
  - Java API cho các máy ảo Java (JVM)

System calls

- System calls gọi hệ thống để sao chép nội dung của tệp này sang tệp khác



System calls

- Ví dụ về API chuẩn

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t     read(int fd, void *buf, size_t count)
```

return value | function name | parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer into which the data will be read
- `size_t count`—the maximum number of bytes to be read into the buffer

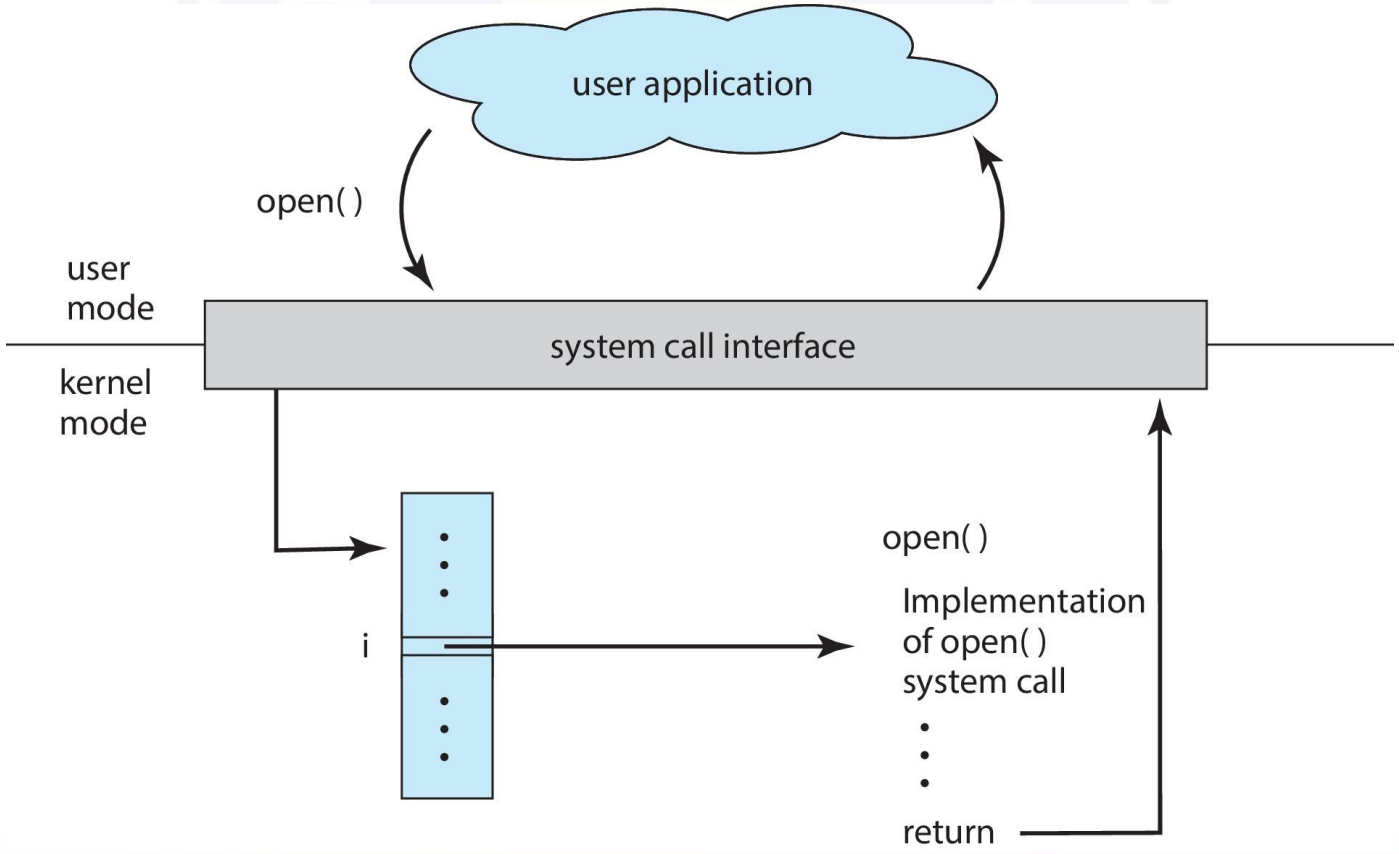
On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.

System calls

- Thực thi System Call
  - Thông thường, một số được liên kết với mỗi System Call
    - Giao diện System Call duy trì một bảng được lập chỉ mục theo các số này
  - Giao diện System Call gọi System Call dự định trong nhân hệ điều hành và trả về trạng thái của cuộc gọi hệ thống cũng như mọi giá trị trả về
  - Người gọi không cần biết gì về cách thực hiện System Call
    - Chỉ cần tuân theo API và hiểu hệ điều hành sẽ làm gì khi có lệnh gọi
    - Hầu hết các chi tiết về giao diện hệ điều hành đều bị API ẩn khỏi lập trình viên
      - Được quản lý bởi thư viện hỗ trợ thời gian chạy (bộ hàm được tích hợp trong thư viện đi kèm với trình biên dịch)

System calls

- Quan hệ giữa API – System Call – OS



System calls

- Đối số trong System Call
  - Thông thường, cần nhiều thông tin hơn là chỉ nhận dạng System Call mong muốn
  - Loại và lượng thông tin chính xác khác nhau tùy theo hệ điều hành và cuộc gọi
  - Ba phương thức chung được sử dụng để truyền tham số cho HĐH

System calls

- Đối số trong System Call
  - Truyền tham số trong các thanh ghi.
    - Đây là phương pháp đơn giản nhất trong ba phương pháp
    - Ở đây chúng ta truyền trực tiếp các tham số vào các thanh ghi.
    - Nhưng nó sẽ bị hạn chế khi số lượng tham số lớn hơn số lượng thanh ghi.

System calls

- Đối số trong System Call
  - Truyền tham số trong các thanh ghi.
    - Đây là mã chương trình C:

```c
// Passing parameters in registers.

#include <fcntl.h>
#include <stdio.h>

int main()
{
    const char* pathname = "example.txt";
    int flags = O_RDONLY;
    mode_t mode = 0644;

    int fd = open(pathname, flags, mode);
  // in function call open(), we passed the parameters pathanme,flags,mode to the kernal directly

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    // File operations here...

    close(fd);
    return 0;
}
```

System calls

- Đối số trong System Call
  - Địa chỉ của khối được truyền dưới dạng tham số
    - Nó có thể được áp dụng khi số lượng tham số lớn hơn số lượng thanh ghi.
    - Các tham số được lưu trữ trong các khối hoặc bảng.
    - Địa chỉ của khối được chuyển đến thanh ghi dưới dạng tham số.
    - Được sử dụng phổ biến nhất trong Linux và Solaris.

System calls

- Đối số trong System Call
  - Địa chỉ của khối được truyền dưới dạng tham số
    - Đây là mã chương trình C:

```c
//Address of the block is passed as parameters

#include <stdio.h>
#include <fcntl.h>

int main() {
    const char *pathname = "example.txt";
    int flags = O_RDONLY;
    mode_t mode = 0644;

    int params[3];
          // Block of data(parameters) in array
    params[0] = (int)pathname;
    params[1] = flags;
    params[2] = mode;

    int fd = syscall(SYS_open, params);
          // system call

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    // File operations here...

    close(fd);
    return 0;
}
```

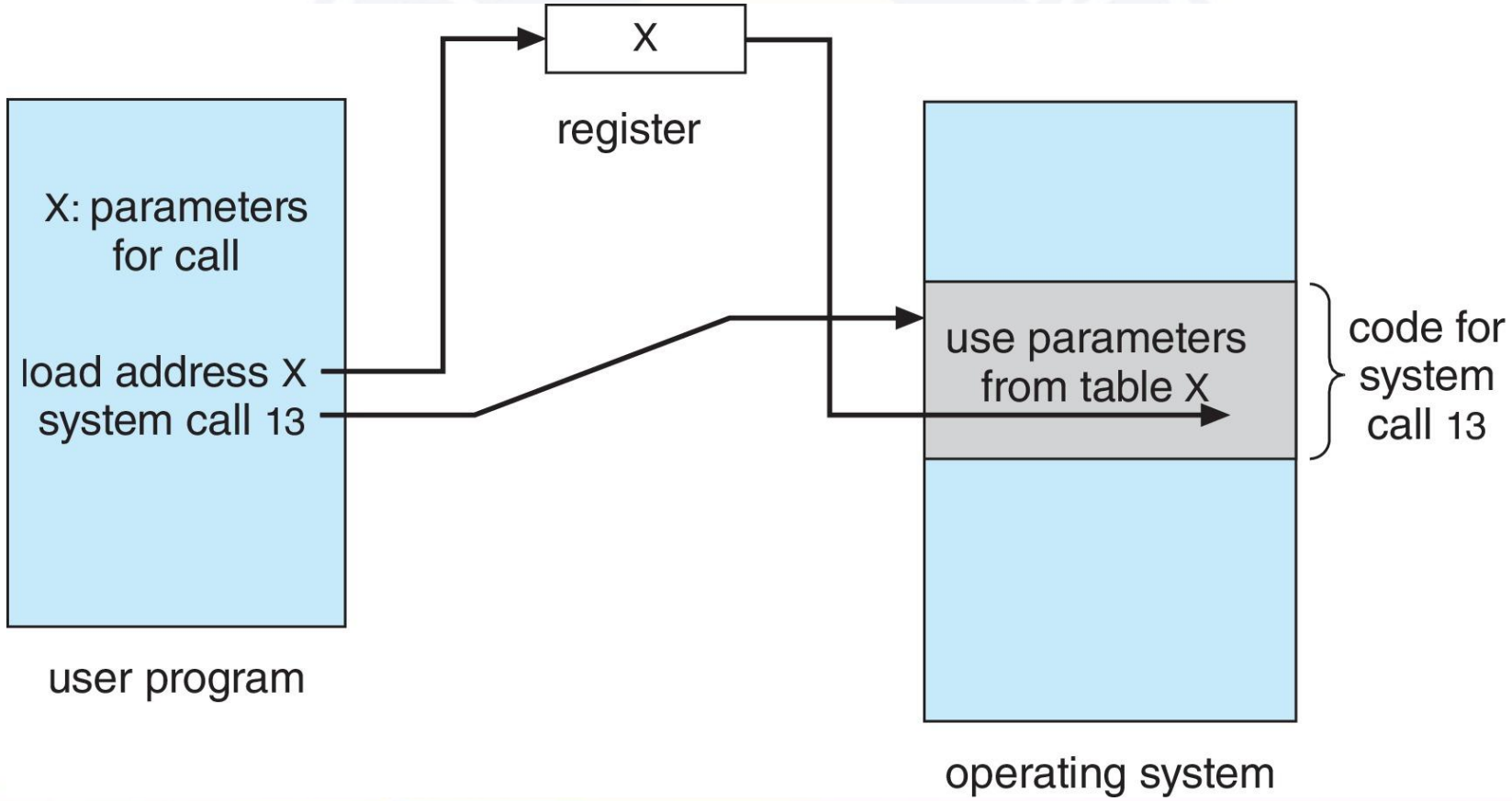
System calls

- Đối số trong System Call
  - Các tham số được đẩy vào một ngăn xếp
    - Trong phương pháp này, các tham số có thể được đẩy vào bằng chương trình và xuất hiện bằng hệ điều hành
    - Vì vậy Kernal có thể dễ dàng truy cập dữ liệu bằng cách lấy thông tin từ đầu ngăn xếp.
    - Đây là mã chương trình C

```c
//parameters are pushed into the stack

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    const char *pathname = "example.txt";
    int flags = O_RDONLY;
    mode_t mode = 0644;

    int fd;
    asm volatile(
        "mov %1, %%rdi\n"
        "mov %2, %%rsi\n"
        "mov %3, %%rdx\n"
        "mov $2, %%rax\n"
        "syscall"
        : "=a" (fd)
        : "r" (pathname), "r" (flags), "r" (mode)
        : "%rdi", "%rsi", "%rdx"
    );

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    // File operations here...

    close(fd);
    return 0;
}
```

System calls

- Truyền tham số qua bảng



System calls

- Các loại System Calls
  - Điều khiển tiến trình
    - Tạo tiến trình, kết thúc tiến trình
    - Hoàn thành, hủy bỏ
    - Tải, thực thi
    - Thiết lập thuộc tính tiến trình
    - Thời gian chờ
    - Chờ sự kiện, chờ tín hiệu
    - Cấp phát bộ nhớ

System calls

- Các loại System Calls
  - Quản lý tập tin
    - Tạo, xóa
    - Mở, đóng
    - Đọc, ghi, đặt lại vị trí
    - Nhận và thiết lập các thuộc tính tập tin

System calls

- Các loại System Calls
  - Quản lý thiết bị
    - Yêu cầu, trả lại
    - Đọc, ghi, đặt lại vị trí
    - Thiết lập thuộc tính thiết bị

System calls

- Các loại System Calls
  - Duy trì thông tin
    - Tiếp nhận thời gian và ngày, thiết lập thời gian và ngày
    - Tiếp nhận dữ liệu hệ thống, thiết lập dữ liệu hệ thống
    - Nhận và thiết lập tiến trình, tập tin và thuộc tính thiết bị

System calls

- Các loại System Calls
  - Liên lạc
    - Tạo, xóa kết nối liên lạc
    - Gởi, nhận tin theo cơ chế message passing
    - Tạo và truy cập vào bộ nhớ theo cơ chế Shared-memory
    - Chuyển đổi trạng thái thông tin

System calls

- Các loại System Calls
  - Bảo vệ
    - Điều khiển truy cập tài nguyên
    - Nhận và thiết lập quyền truy cập
    - Cho phép và cấm người dùng truy cập

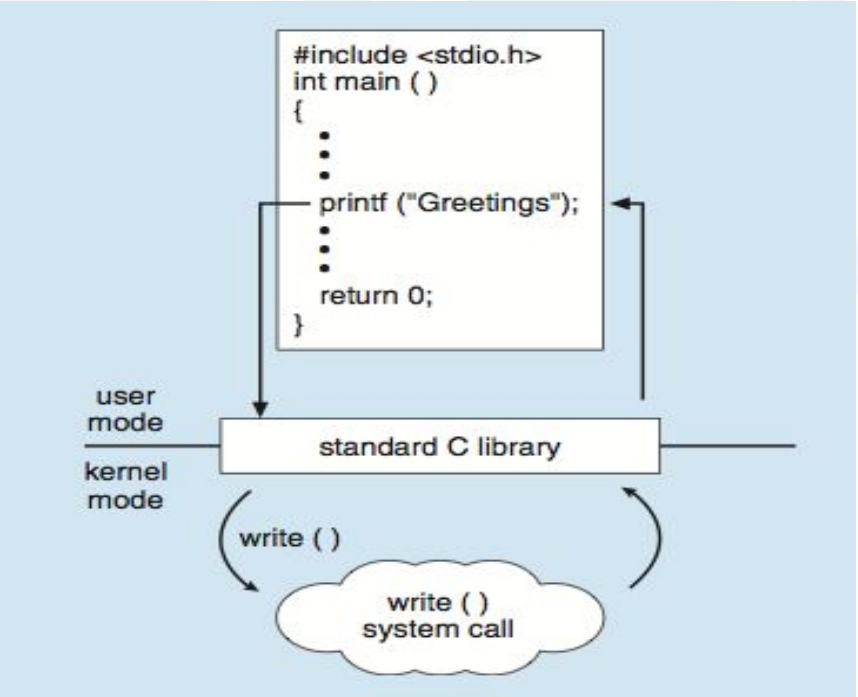
System calls

- System Calls trong Windows và Linux

	Windows	Unix
Process Control	CreateProcess()	fork()
	ExitProcess()	exit()
	WaitForSingleObject()	wait()
File Manipulation	CreateFile()	open()
	ReadFile()	read()
	WriteFile()	write()
	CloseHandle()	close()
Device Manipulation	SetConsoleMode()	ioctl()
	ReadConsole()	read()
	WriteConsole()	write()
Information Maintenance	GetCurrentProcessID()	getpid()
	SetTimer()	alarm()
	Sleep()	sleep()
Communication	CreatePipe()	pipe()
	CreateFileMapping()	shmget()
	MapViewOfFile()	mmap()
Protection	SetFileSecurity()	chmod()
	InitlializeSecurityDescriptor()	umask()
	SetSecurityDescriptorGroup()	chown()

System calls

- Thư viện C chuẩn
  - Chương trình C gọi hàm thư viện **printf()**, hàm thực hiện lời gọi hệ thống **write()**

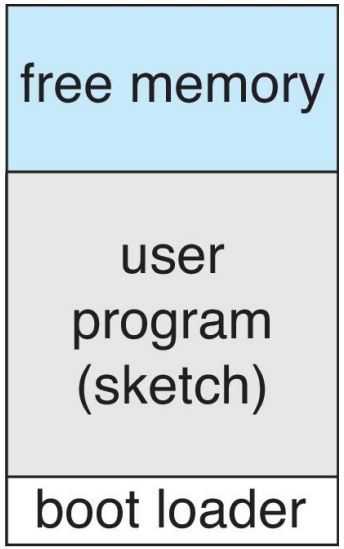


System calls

- Ví dụ: Arduino
  - Đơn nhiệm
  - Không có hệ điều hành
  - Các chương trình (bản phác thảo) được tải qua USB vào bộ nhớ flash
  - Không gian bộ nhớ đơn
  - Chương trình tải bootloader
  - Thoát chương trình -> tải lại shell

(a)

At system startup

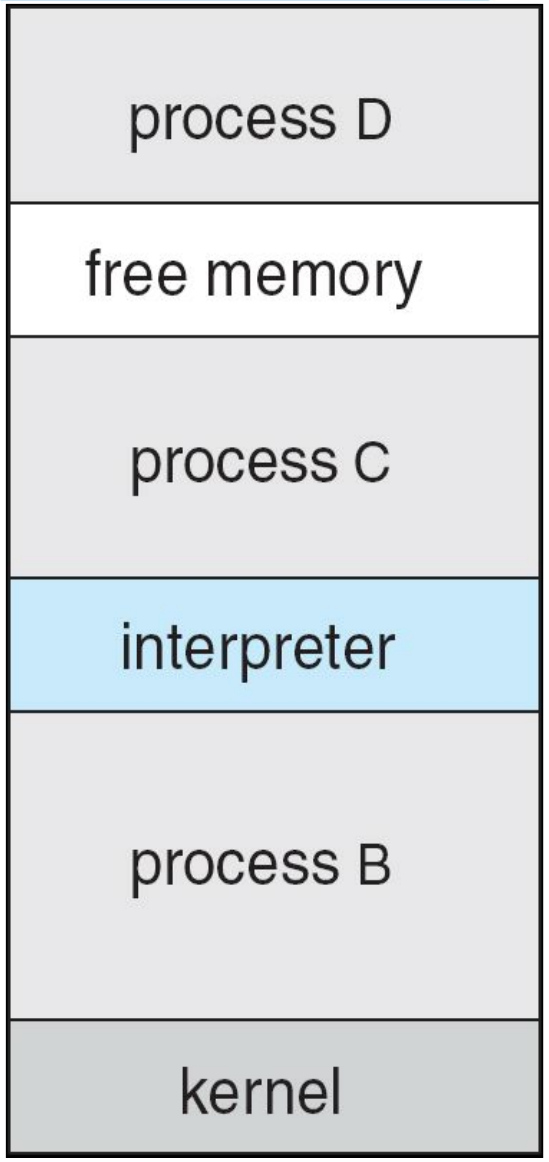


(b)

running a program

System calls

- Ví dụ: FreeBSD
  - Biến thể của Unix
  - Đa nhiệm
  - Khi người dùng đăng nhập -> chọn Shell
  - Shell thực thi lời gọi hệ thống fork() để tạo tiến trình
    - Thực thi exec() để đưa chương trình vào tiến trình
    - Shell chờ tiến trình kết thúc hoặc tiếp tục với các lệnh khác của người dùng
  - Khi tiến trình thoát:
    - Code=0 : không có lỗi
    - Code>0 : có lỗi

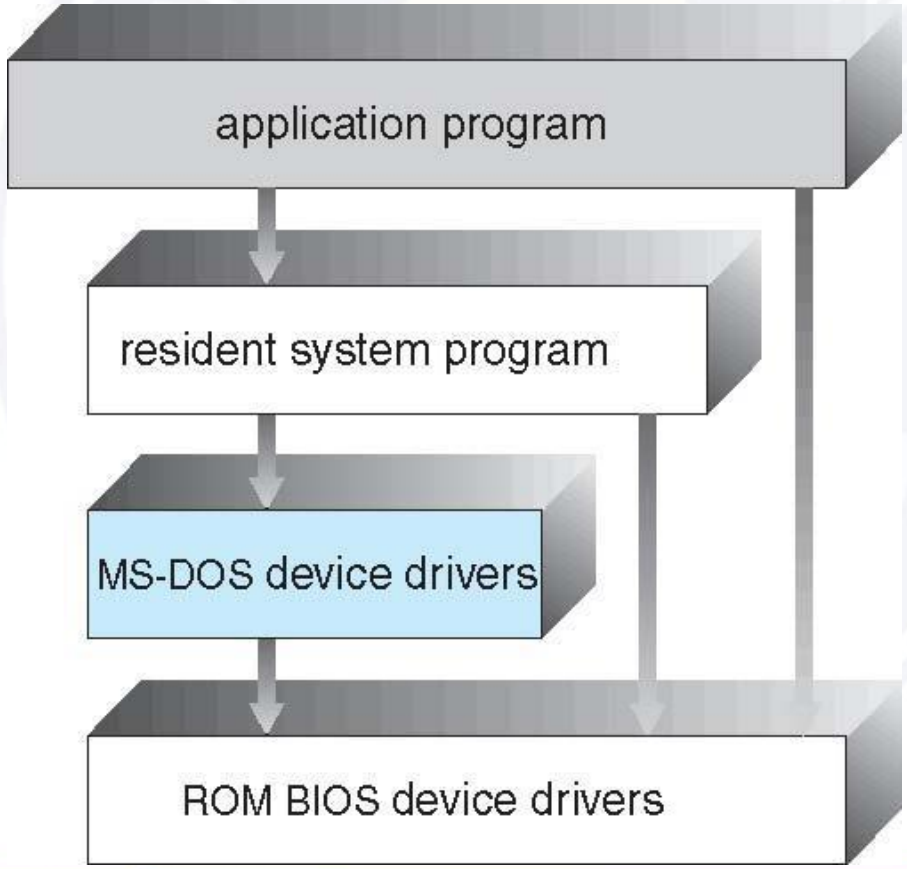


Cấu trúc hệ điều hành

- Bao gồm các cấu trúc
  - Cấu trúc đơn giản – MS-DOS
  - Phức tạp hơn – UNIX
  - Cấu trúc phân lớp
  - Cấu trúc Microkernel

Cấu trúc hệ điều hành

- Cấu trúc đơn giản – MS-DOS

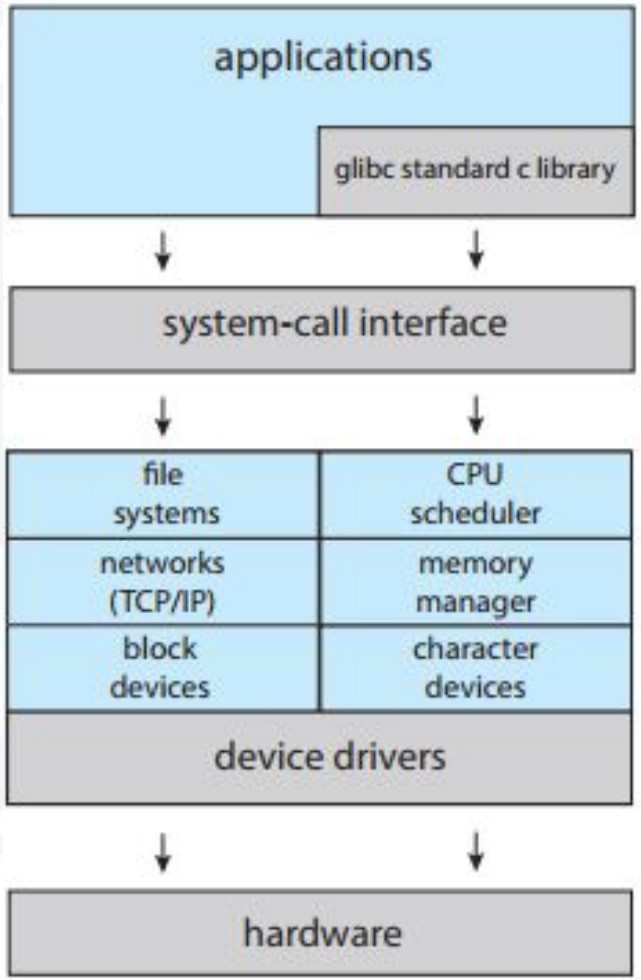


Cấu trúc hệ điều hành

- Cấu trúc nguyên khối – UNIX gốc
  - UNIX - giới hạn bởi chức năng phần cứng, bản gốc của hệ điều hành UNIX có cấu trúc hạn chế. Các hệ điều hành UNIX bao gồm hai phần riêng biệt:
    - Chương trình hệ thống
    - Nhân (Kernel)
      - Bao gồm tất cả mọi thứ bên dưới giao diện lời gọi hệ thống và bên trên phần cứng vật lý
      - Cung cấp hệ thống file, lập lịch CPU, quản lý bộ nhớ, và các chức năng khác

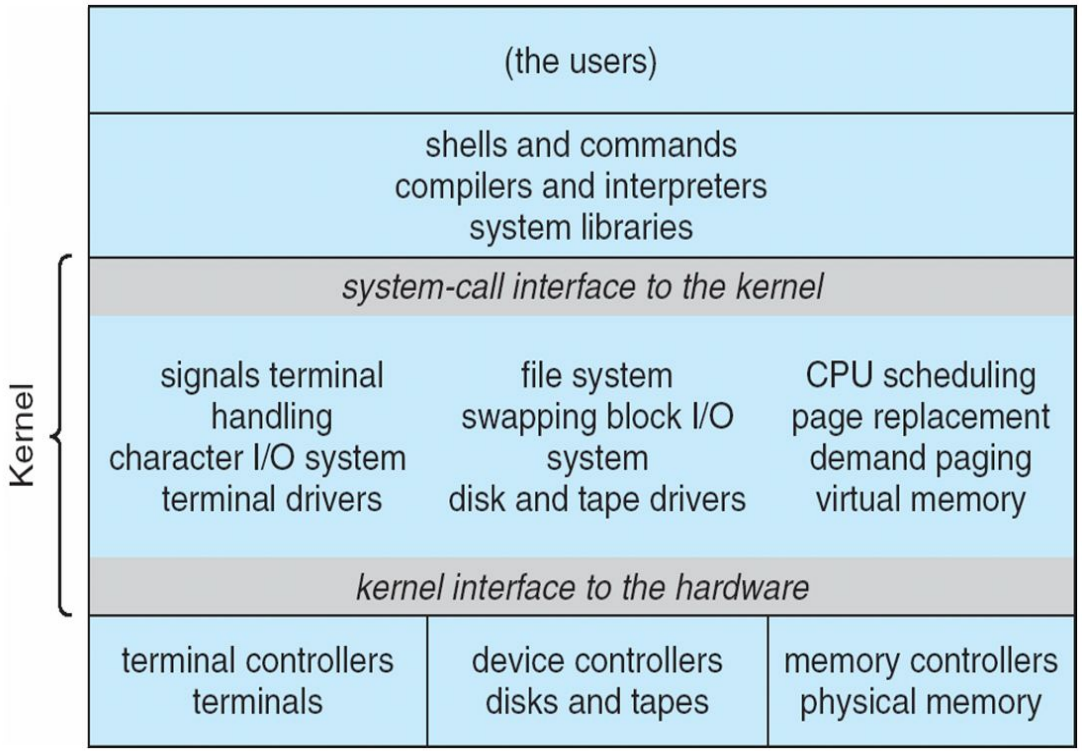
Cấu trúc hệ điều hành

- Cấu trúc hệ thống Linux
  - Thiết kế nguyên khối cộng với mô-đun



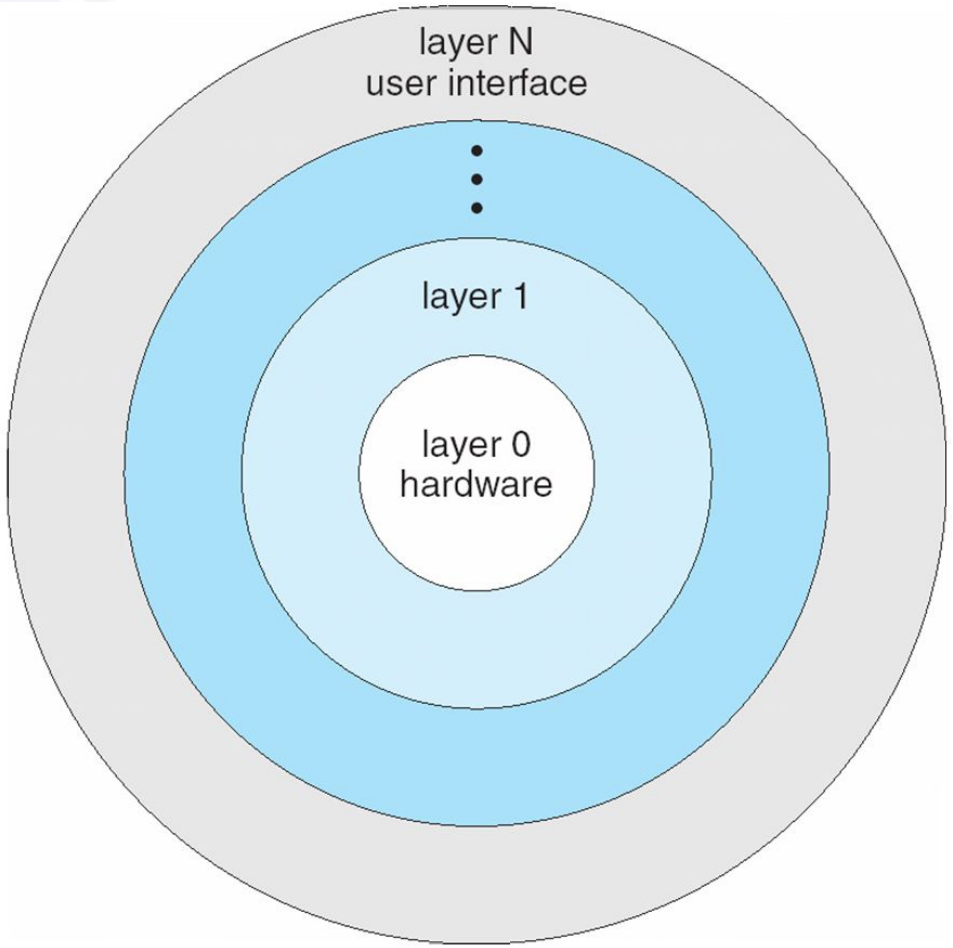
Cấu trúc hệ điều hành

- Cấu trúc hệ thống UNIX truyền thống
  - Vượt ra cấu trúc đơn giản nhưng chưa phân lớp đầy đủ



Cấu trúc hệ điều hành

- Cấu trúc phân tầng
  - Hệ điều hành được chia thành một số lớp (cấp độ), mỗi lớp được xây dựng dựa trên lớp thấp hơn. Lớp dưới cùng (layer 0) là phần cứng; lớp cao nhất (lớp N) là giao diện người dùng
  - Với tính đơn thể hóa, mỗi lớp chỉ sử dụng các dịch vụ và chức năng của lớp thấp hơn



Cấu trúc hệ điều hành

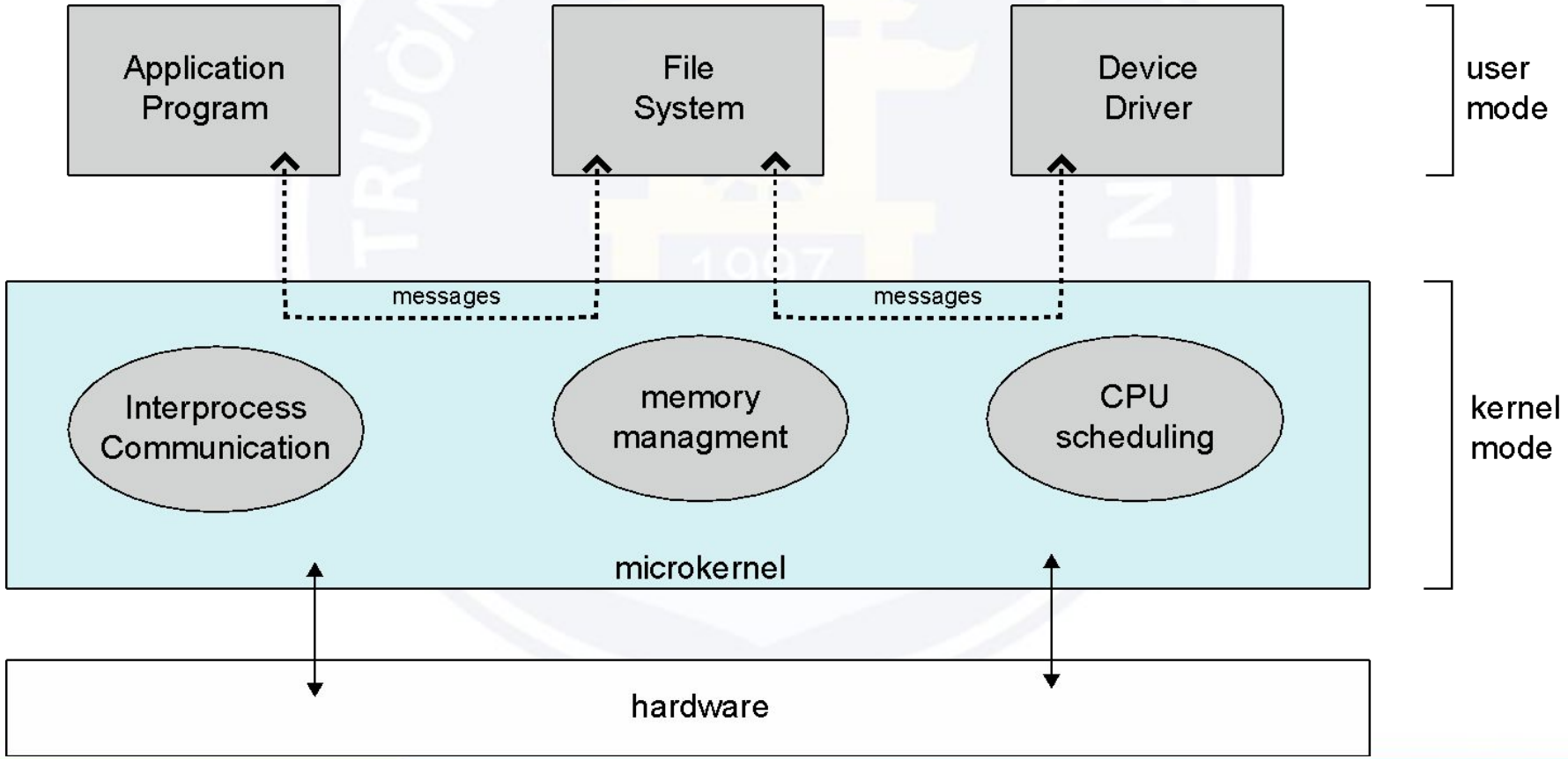
- Cấu trúc Microkernel
  - Di chuyển nhiều thành phần từ chế độ nhân sang chế độ người dùng
  - HĐH Mach là ví dụ cho cấu trúc Microkernel
    - Mac OS X kernel (Darwin) một phần dựa trên Mach
  - Liên lạc giữa các module người dùng sử dụng cơ chế message passing

Cấu trúc hệ điều hành

- Cấu trúc Microkernel
  - Lợi ích:
    - Dễ dàng mở rộng
    - Dễ dàng chuyển HĐH sang một kiến trúc mới
    - Đáng tin cậy hơn (ít mã lệnh chạy trong kernel mode)
    - An toàn hơn
  - Hạn chế
    - Việc liên lạc từ chế độ người dùng sang chế độ nhân tốn hiệu suất cao

Cấu trúc hệ điều hành

- Cấu trúc Microkernel

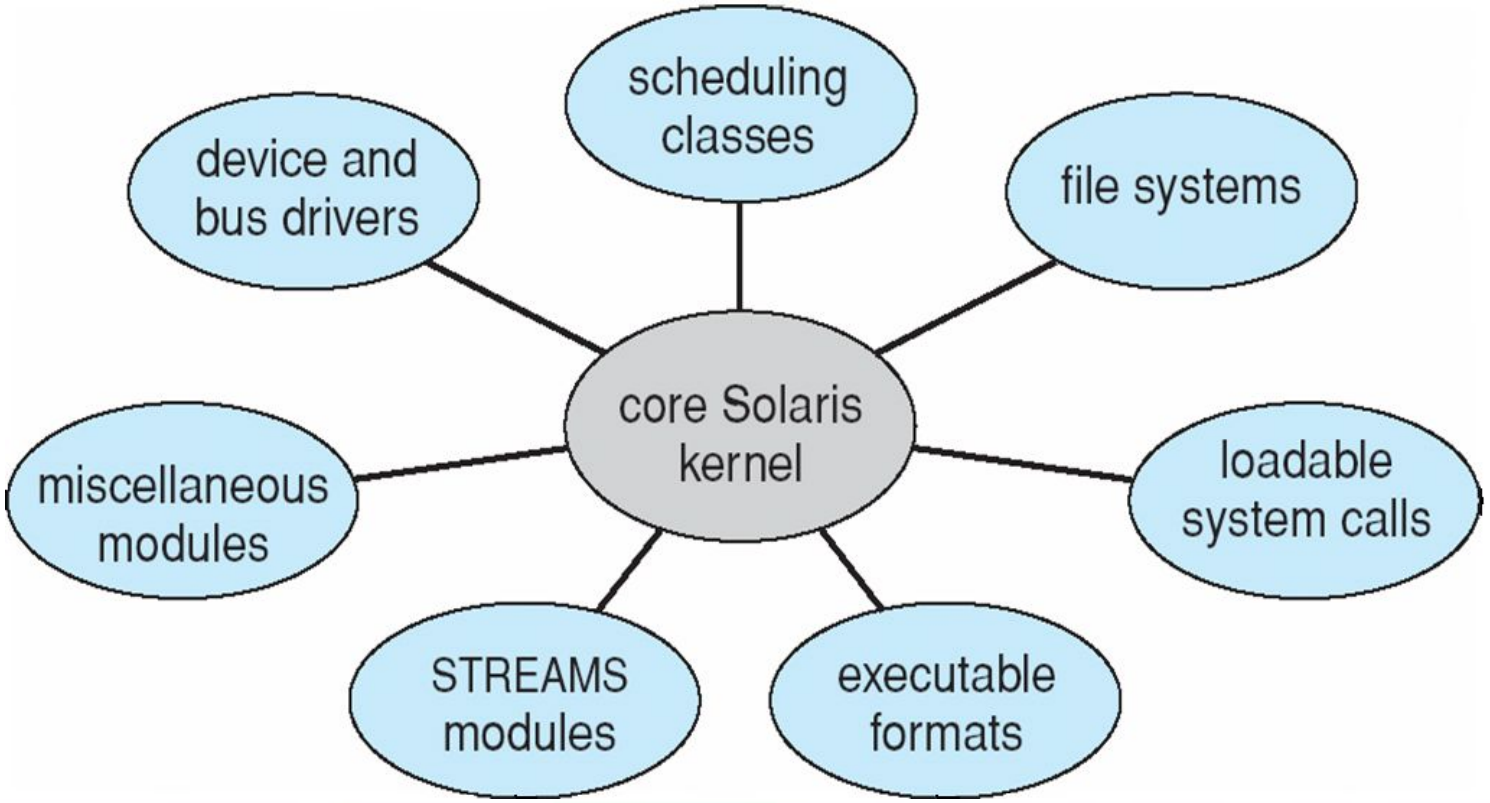


Cấu trúc hệ điều hành

- Cấu trúc Module
  - Nhiều HĐH hiện đại được thực thi theo cấu trúc module hóa hạt nhân
    - Sử dụng cách tiếp cận hướng đối tượng
    - Mỗi thành phần lõi riêng biệt
    - Mỗi thành phần lõi liên lạc với các thành phần khác qua giao diện
    - Mỗi thành phần lõi có thể được nạp thông qua nhân chính
  - Giống cấu trúc phân lớp nhưng linh hoạt hơn
    - Linux, Solaris, …

Cấu trúc hệ điều hành

- Cấu trúc Module - Solaris

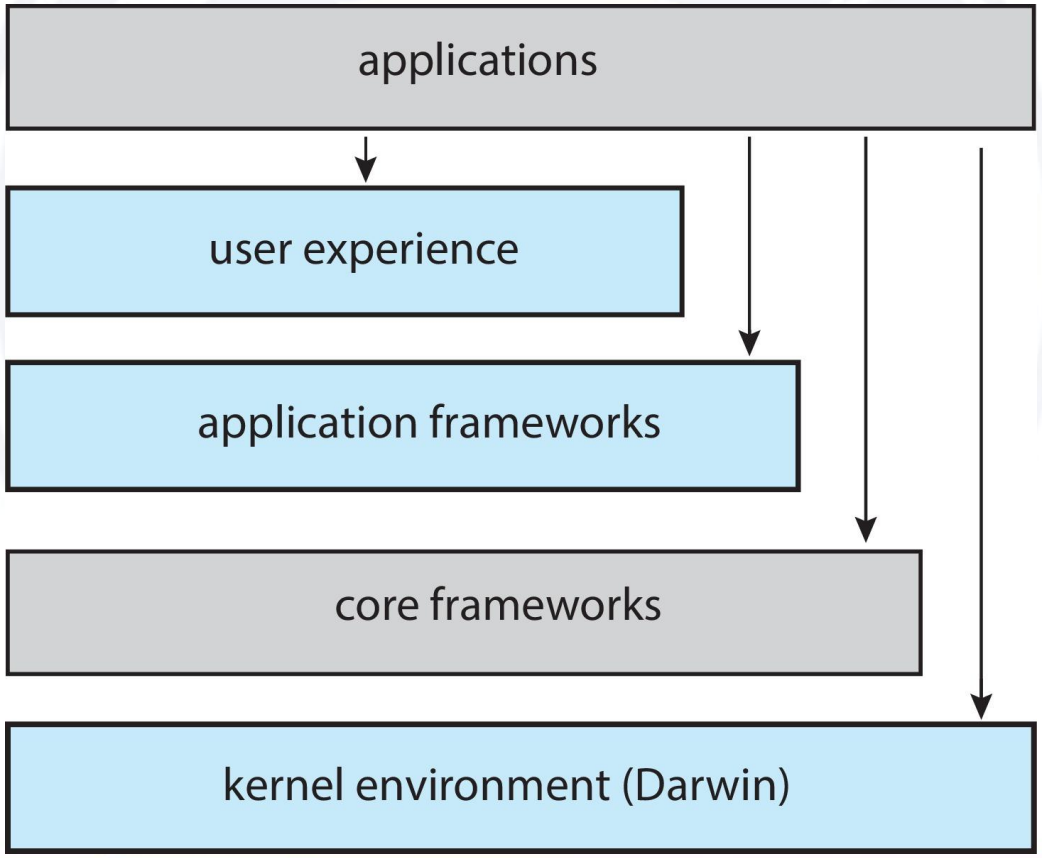


Cấu trúc hệ điều hành

- Các hệ thống lai
  - Hầu hết các hệ điều hành hiện đại thực sự không sử dụng một mô hình thuần túy
    - Mô hình lai sử dụng nhiều hướng tiếp cận để giải quyết tính hiệu quả, an ninh, tính khả dụng
    - Nhân của Linux và Solaris được thiết kế nguyên khối, cộng với các mô đun chức năng động
    - Windows hầu hết là nguyên khối, cộng với microkernel của các hệ thống phụ khác nhau – personalities
  - Apple Mac OS X là hệ thống lai, phân lớp, giao diện người dung Aqua và môi trường lập trình Cocoa

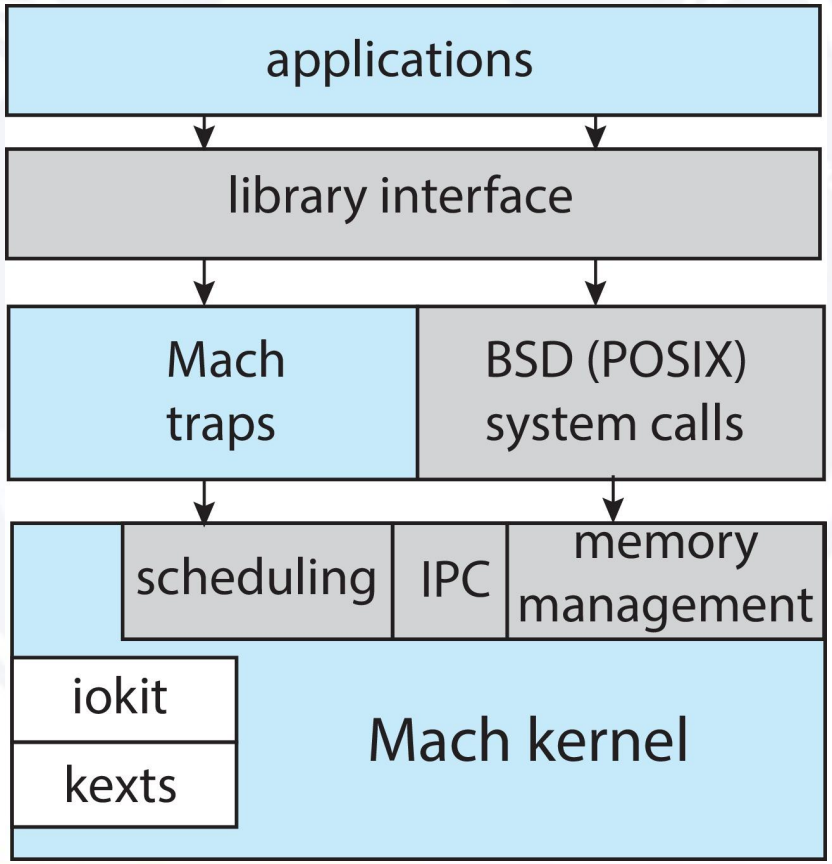
Cấu trúc hệ điều hành

- Kiến trúc hệ điều hành macOS và iOS của Apple



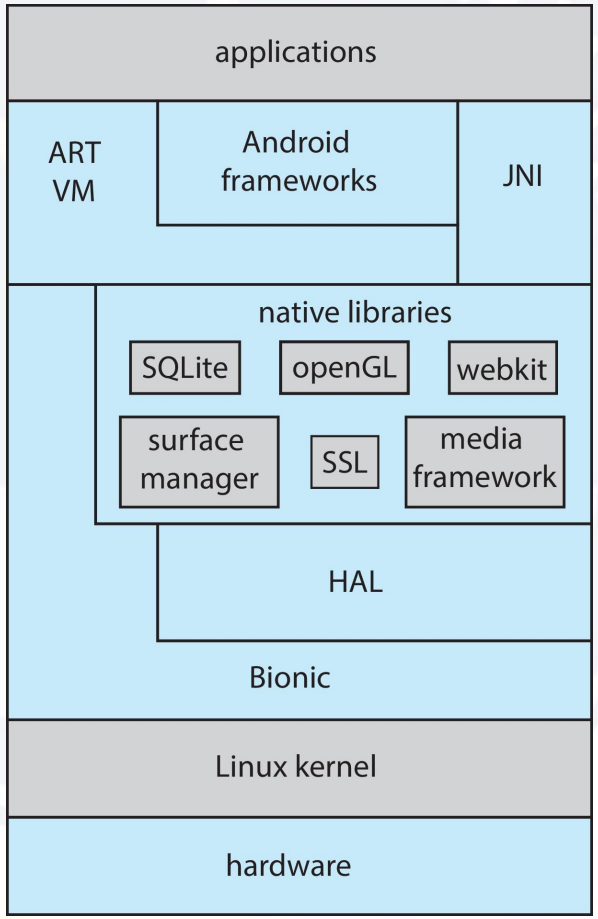
Cấu trúc hệ điều hành

- Kiến trúc của Darwin



Cấu trúc hệ điều hành

- Kiến trúc của Android



Các chương trình hệ thống

- Chương trình hệ thống cung cấp một môi trường thuận lợi cho việc phát triển chương trình và thực thi. Chúng có thể được chia thành
  - Quản lý File
  - Chương trình quản lý trạng thái thông tin
  - Cập nhật File
  - Ngôn ngữ lập trình

Các chương trình hệ thống

- Chương trình hệ thống cung cấp một môi trường thuận lợi cho việc phát triển chương trình và thực thi. Chúng có thể được chia thành
  - Chương trình tải và thực thi
  - Liên lạc
  - Dịch vụ nền
  - Chương trình ứng dụng

Các chương trình hệ thống

- Quản lý File – Tạo, xóa, sao chép, đổi tên, in ấn, kết xuất, liệt kê và các thao tác khác trên tập tin và thư mục
- Chương trình quản lý trạng thái thông tin
  - Một số yêu cầu hệ thống về ngày, giờ, số lượng bộ nhớ có sẵn, không gian đĩa, số lượng người dùng
  - Một số khác cung cấp hiệu suất, việc đăng nhập, và các trình gỡ rối
  - Các chương trình này định dạng và xuất thông tin ra màn hình hoặc thiết bị xuất khác
  - Một số hệ thống thực thi registry – đối tượng được sử dụng để lưu trữ và lấy thông tin cấu hình

Các chương trình hệ thống

- Cập nhật File
  - Trình soạn thảo văn bản giúp tạo và cập nhật tập tin
  - Các lệnh đặc biệt để tìm kiếm nội dung của các tập tin hoặc thực hiện chuyển đổi văn bản
- Ngôn ngữ lập trình – trình biên dịch, trình gỡ rối được cung cấp
- Chương trình tải và thực thi – trình nạp, trình liên kết, trình gỡ rối dùng cho ngôn ngữ bậc cao và ngôn ngữ máy

Các chương trình hệ thống

- Liên lạc - Cung cấp cơ chế cho việc tạo ra các kết nối ảo giữa các tiến trình, người dùng, và hệ thống máy tính
  - Cho phép người dùng gửi tin nhắn đến một màn hình của người khác, duyệt web, gửi mail, đăng nhập từ xa, chuyển các tập tin từ một máy tới máy khác

Các chương trình hệ thống

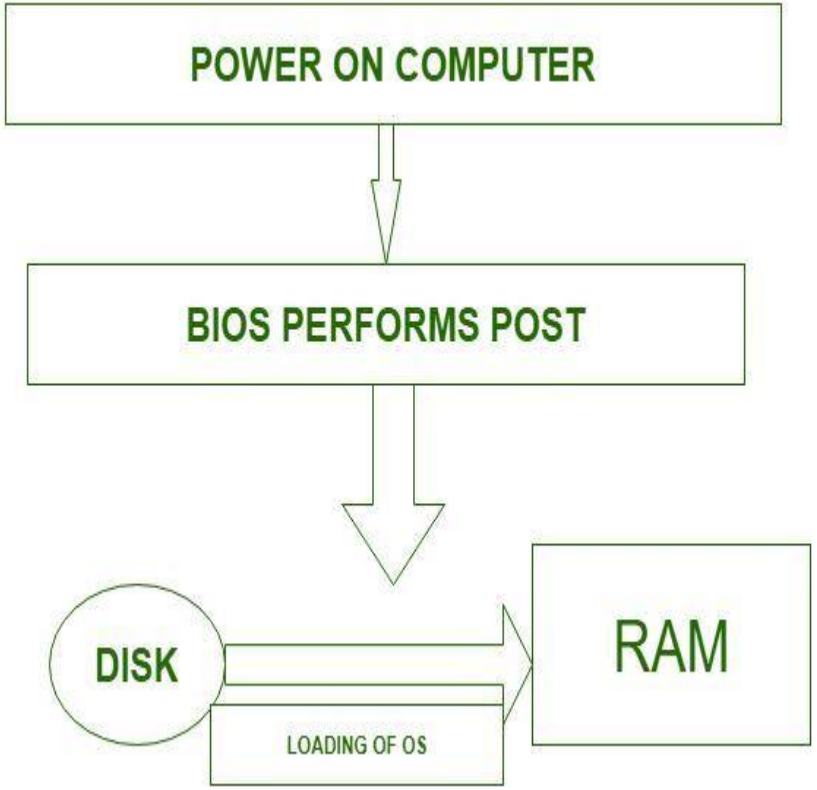
- Dịch vụ nền
  - Chạy lúc khởi động
    - Một số chạy tại lúc hệ thống khởi động và lúc hệ thống kết thúc
    - Một số chạy từ lúc khởi động đến lúc tắt máy
  - Cung cấp các tiện ích như kiểm tra đĩa, điều phối tiến trình, báo lỗi đăng nhập, in ấn
  - Chạy trong chế độ người dùng, không chạy trong chế độ hạt nhân

Các chương trình hệ thống

- Chương trình ứng dụng
  - Không thuộc về hệ thống
  - Chạy bởi người dùng
  - Không được coi là một phần của hệ điều hành
  - Được khởi động bằng lệnh, chuột, cảm ứng tay

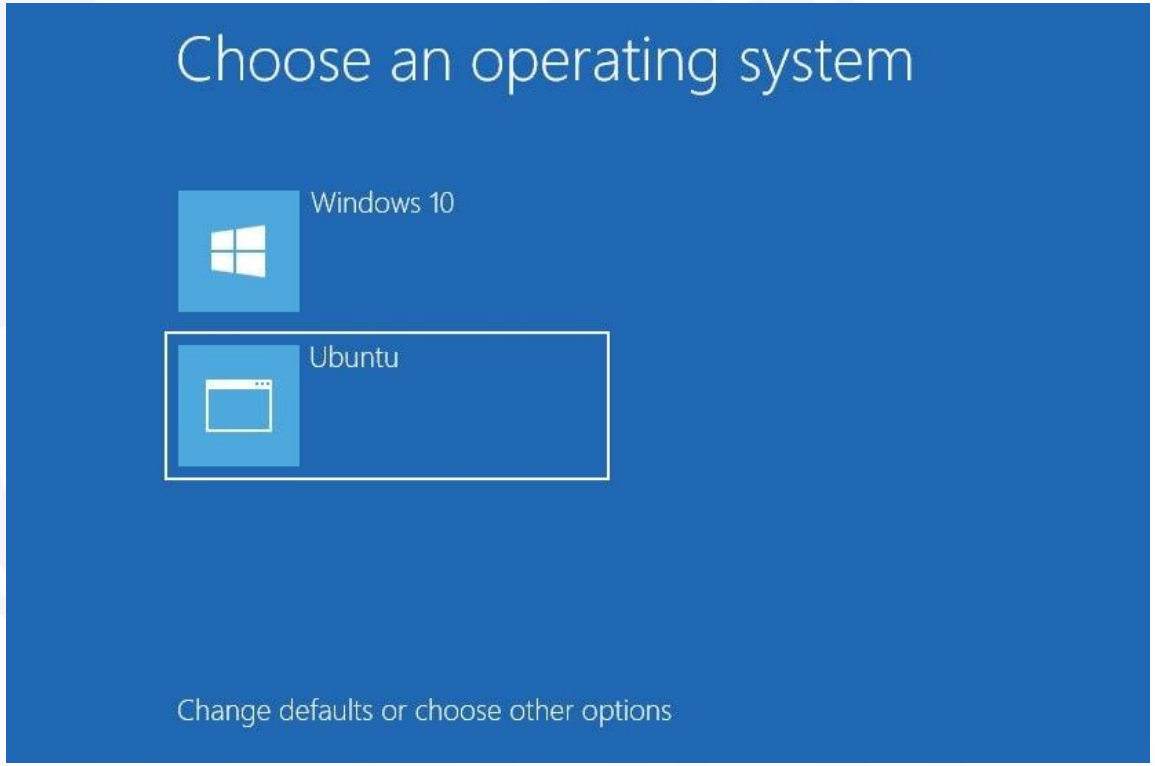
Boot hệ thống

- Booting
  - Khi hệ thống máy tính được khởi động, sẽ có một cơ chế trong hệ thống tải hệ điều hành từ bộ lưu trữ thứ cấp vào bộ nhớ chính hoặc RAM của hệ thống. Đây được gọi là quá trình khởi động của hệ thống.
  - Có hai kiểu khởi động tùy thuộc vào số lượng hệ điều hành được cài đặt trên máy/máy tính.
    - Booting
    - Dual Booting



Boot hệ thống

- Dual Booting
  - Khi hai hệ điều hành được cài đặt trên một hệ thống máy tính, nó được gọi là khởi động kép.



THANK YOU!

665-667-669 Điện Biên Phủ, P. 1, Q. 3, Tp. HCM

1800 1568

www.vhu.edu.vn